Edtech Platform, Agent Tooling and Voice Unit: Technical Design

Oct 24, 2026 · @Lekha

1. Purpose and scope

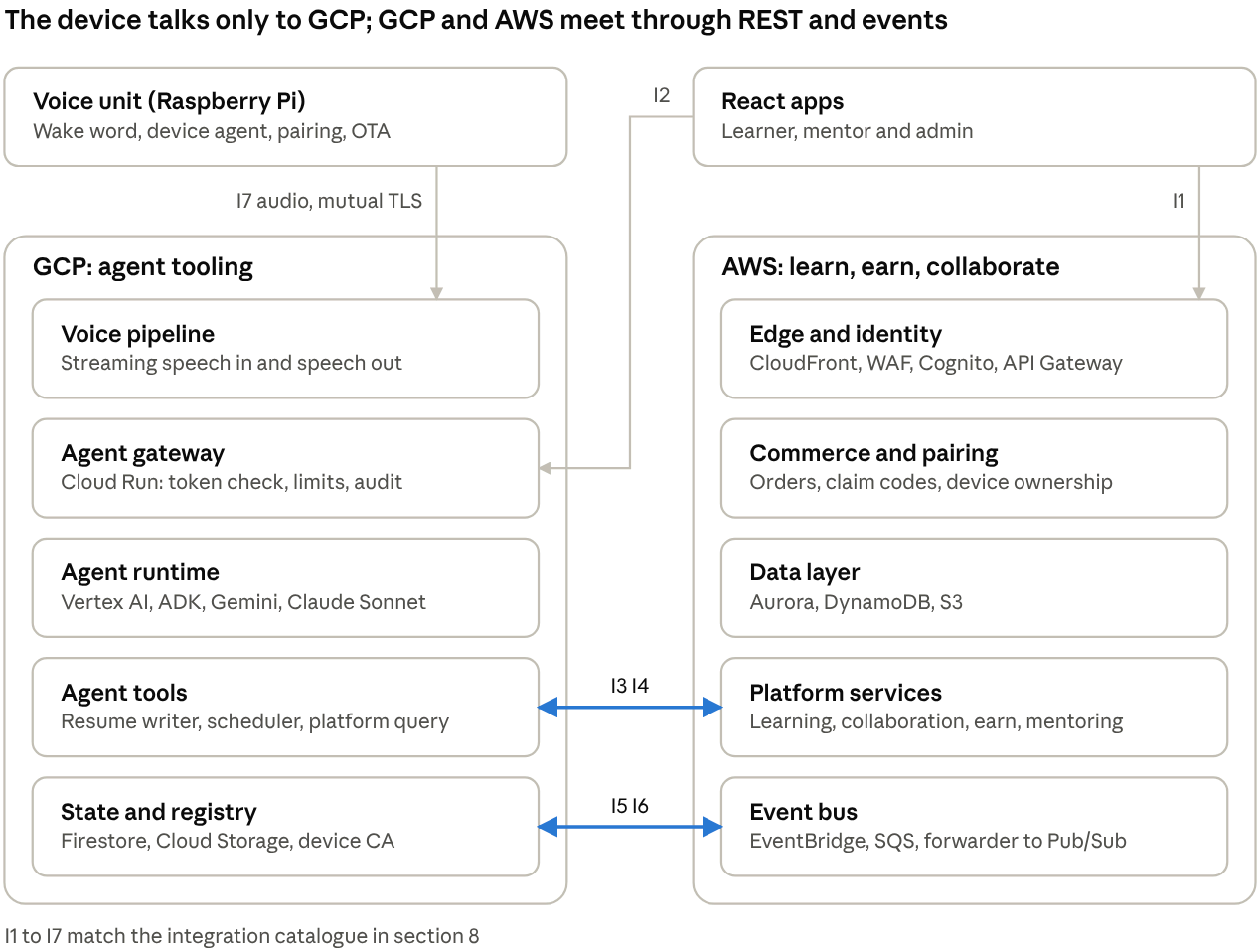
This document specifies one system made of three products: an edtech platform on AWS, an agent service on GCP, and a Raspberry Pi voice unit that talks to the agents. Backends are Python and every user interface is React.

Area	What it is	Cloud	Main users
1. Learn-and-earn platform	Courses, collaboration, paid tasks, mentoring, profiles	AWS	Students, mentors, admins
2. Agent tooling	Resume writer, mentor-mentee scheduler, shared agent runtime	GCP	Students, mentors, and the platform itself
3. Voice unit	A purchasable device that behaves like a smart speaker and uses the same agents	Device, with GCP as its only backend	Students and mentors at home

Out of scope for this version: the hardware bill of materials and manufacturing, native mobile apps, and payment-provider selection. They are listed as open questions in section 16.

2. Architecture overview

The AWS platform owns data and money, the GCP agents do the thinking and the talking, and the device is a thin client of GCP. The two clouds meet in two places only: REST calls with a user token, and events.



system architecture · three areas, two clouds

How to read it: users reach the platform through the React apps (I1) and reach the agents through the agent gateway (I2). The voice unit streams to the GCP voice pipeline (I7). Agents read and write platform data through the platform API (I3, I4), and the two event buses exchange domain events (I5, I6). The device never calls AWS, so there is a single place to authenticate, rate-limit and audit device traffic.

3. Technology stack

Python 3.12 for every backend and device service, React with TypeScript for every user interface, and infrastructure defined as code on both clouds.

Layer	Choice	Notes
Backend language	Python 3.12	One language across AWS services, GCP agents and the device client
API framework	FastAPI with Pydantic v2	OpenAPI generated for every service; shared schema package
Async and jobs	asyncio, Celery or AWS Lambda workers	Webhooks, payouts, PDF rendering
Agent framework	Google Agent Development Kit (Python) on Vertex AI, with an LLM adapter	Gemini by default; Claude Sonnet 5.5 behind the same adapter for resume quality
Web UI	React 18, TypeScript, Vite, TanStack Query, Tailwind	One design-system package shared by all apps
Web apps	Learner app, mentor app, admin console	Separate React apps in one monorepo
Device UI	Optional React status page served by the Pi on the local network	Used for Wi-Fi setup and pairing; the voice path has no screen
Real-time	WebSockets through API Gateway or AppSync	Chat, presence, live collaboration
Infrastructure as code	Terraform for both clouds	One state per environment
CI/CD	GitHub Actions with OIDC to AWS and GCP	No long-lived cloud keys in CI
Containers	Docker, ECS Fargate on AWS, Cloud Run on GCP	Same image build for both clouds where code is shared
Observability	OpenTelemetry in every Python service	One trace ID across clouds and device

A monorepo layout keeps contracts in one place: `/packages/contracts` (OpenAPI and event schemas), `/services/*` (AWS), `/agents/*` (GCP), `/device/*` (Pi), `/web/*` (React apps), `/infra/*` (Terraform).

4. Area 1: AWS learn-and-earn platform

The platform is the system of record for users, learning, money and meetings. It runs as small FastAPI services on ECS Fargate behind one API Gateway, and publishes domain events that GCP agents consume.

Component	AWS service	Responsibility
Web delivery	S3, CloudFront, WAF	Hosts the React apps, caches content, filters abuse
Identity	Cognito user pools	Sign-up, sign-in, roles (student, mentor, admin), tokens
API front door	API Gateway (HTTP and WebSocket)	Routing, JWT validation, throttling
Learning service	ECS Fargate (FastAPI)	Courses, lessons, progress, certificates
Collaboration service	ECS Fargate, WebSocket API	Groups, chat, shared project spaces, presence
Earn service	ECS Fargate	Tasks and bounties, submissions, review, wallet ledger, payouts
Mentoring service	ECS Fargate	Mentor profiles, availability, meeting records, feedback
Profile and skills service	ECS Fargate	Profiles, skills graph, resume source data
Commerce service	ECS Fargate	Voice-unit orders, claim codes, device ownership
Search	OpenSearch Serverless	Course and mentor search (add after launch if needed)
Relational data	Aurora PostgreSQL	Users, enrolments, ledger, orders, meetings
Key-value data	DynamoDB	Chat messages, activity feed, idempotency keys
Files	S3	Course content, uploads, signed resume copies

Component	AWS service	Responsibility
Events	EventBridge, SQS with dead-letter queues	Domain events and reliable delivery
Secrets and keys	Secrets Manager, KMS	Credentials, field encryption
Observability	CloudWatch, X-Ray via OpenTelemetry	Logs, metrics, traces

Design rules for this area:

- Each service owns its tables; no service reads another service's tables. Cross-service data moves by API or event.
- The earn service keeps a double-entry ledger in Aurora. Payouts are idempotent and reconciled daily against the payment provider.
- Every state change that agents care about emits an event with a schema in `/packages/contracts`, for example `profile.updated`, `availability.changed`, `meeting.booked`, `task.completed`, `order.paid`.
- Services are stateless; sessions live in Cognito tokens and DynamoDB.

5. Area 2: GCP agent tooling

GCP hosts one agent runtime with a small set of tools. The same agents answer typed requests from the React apps and spoken requests from the voice unit. Agents hold no master data: they read and write profiles, availability and meetings through the AWS platform API.

Component	GCP service	Responsibility
Agent gateway	Cloud Run (FastAPI)	Only public endpoint for agents; validates user tokens, rate-limits, writes audit logs
Agent runtime	Vertex AI Agent Engine with Google ADK	Plans, calls tools, keeps conversation state
Model access	Gemini on Vertex AI; Claude Sonnet 5.5 through the LLM adapter	Model choice per task (see cost section)

Component	GCP service	Responsibility
Resume tool	Cloud Run job or service	Builds a resume from profile data, renders PDF
Scheduler tool	Cloud Run service	Finds common slots, books meetings, sends invites and reminders
Voice pipeline	Cloud Run streaming service with Speech-to-Text and Text-to-Speech	Turns device audio into text and replies back into audio
Session and agent state	Firestore	Conversations, preferences, tool results with a time-to-live
Generated files	Cloud Storage	Resume PDFs and drafts, with signed URLs
Event intake	Pub/Sub push subscription	Receives events forwarded from AWS EventBridge
Device registry	Firestore	Device ID, owner ID, status, certificate fingerprint
Device certificates	Certificate Authority Service	Issues per-device certificates at pairing
Secrets	Secret Manager	Calendar and email credentials, model keys
Observability	Cloud Logging, Cloud Trace via OpenTelemetry	Logs and traces joined to AWS by trace ID

Agent tools

- **Resume writer.** Inputs: profile, skills, completed courses and tasks, target role, template. Output: a structured resume (JSON) and a PDF. The user always reviews and edits the draft in the React app before it is saved; the tool never invents employers, dates or qualifications.
- **Scheduler.** Inputs: mentor and mentee availability, time zones, meeting length, preferences. Output: a booked meeting with a calendar invite and reminders. Booking is confirmed by both sides, and the platform stores the authoritative meeting record.

- **Platform query tool.** Read-only lookups for voice and chat: my tasks, my next meeting, my progress. Write actions need an explicit confirmation turn.

Every tool call carries the end user's token, so agents act with that user's permissions and no more.

6. Area 3: Raspberry Pi voice unit

The unit is a thin client. It listens for a wake word locally, streams audio to the GCP voice pipeline only after the wake word, and plays the spoken reply. The device never calls AWS; the GCP agent gateway is its only endpoint.

Part	Choice	Notes
Board	Raspberry Pi 5 (4 GB) or Pi 4	Pi 5 preferred for on-device wake word and echo cancellation headroom
Audio in	Microphone array HAT or USB array	Beamforming helps in noisy rooms
Audio out	Small powered speaker	Needed for spoken replies
Controls	Hardware mute switch, LED ring	Mute cuts the microphone electrically; LED shows listening and streaming
OS	Raspberry Pi OS Lite, read-only root with an A/B update partition	Safe over-the-air updates and rollback
Wake word	openWakeWord or a licensed engine	Runs on the device; no audio leaves before it fires
Voice activity and echo cancel	Silero VAD, WebRTC audio processing	Ends the utterance cleanly and ignores its own playback
Device agent	Python 3.12 service (asyncio)	Session, streaming client, playback, local cache, health reporting
Transport	gRPC or WebSocket streaming over mutual TLS	Per-device certificate from the GCP certificate authority

Part	Choice	Notes
Setup UI	Small React page served on the local network	Wi-Fi setup and pairing with a claim code
Updates	Signed images from Cloud Storage, announced by Pub/Sub	A/B partitions; automatic rollback on failed health check

Device states

1. Unpaired: serves the setup page and has no cloud access beyond pairing.
2. Paired and idle: wake-word listening only; heartbeat to the registry every few minutes.
3. Streaming: audio flows to the voice pipeline until the utterance ends.
4. Speaking: plays the reply; barge-in stops playback when the wake word is heard again.
5. Suspended or revoked: the registry marks the device inactive and the gateway refuses it.

Offline behaviour: the unit says it has no connection and does nothing else. A local fallback assistant is not part of this version.

7. Identity, security and privacy

One identity (Cognito) is trusted by both clouds, and every cross-cloud and device call is authenticated in both directions.

Concern	Design
User identity	Cognito issues OIDC tokens. The GCP agent gateway validates them against Cognito's public keys.
Agent acting for a user	The user's token travels with the request. For voice, the gateway obtains a short-lived, user-scoped token from AWS using the device's owner link.
GCP calling AWS	Workload identity: the agent service exchanges its Google identity for a short-lived AWS role; no stored AWS keys.

Concern	Design
AWS calling GCP	Signed requests with a service identity; events go through a forwarder to a Pub/Sub push endpoint that checks a signature.
Device identity	Per-device X.509 certificate issued at pairing; mutual TLS to the voice gateway; revocation list checked on connect.
Authorization	Role-based on the platform (student, mentor, admin). Agents get a narrow tool scope per request.
Sensitive actions	Payouts and account changes by voice require an in-app confirmation or a spoken PIN.
Data at rest	KMS and Cloud KMS encryption; resume and meeting data encrypted per user where practical.
Data in transit	TLS 1.2 or higher everywhere; mutual TLS for devices.
Network	WAF at the edge; private subnets for data stores; restrict the GCP gateway to expected callers where possible.
Audit	Every agent tool call logged with user, device, tool, outcome and trace ID.

Privacy of the voice unit:

- The wake word runs on the device. Nothing is sent before it fires.
- Audio is processed in memory and not stored by default. Transcripts are kept only for the conversation window unless the user opts in.
- A hardware mute switch and a visible LED show when the microphone is off or streaming.
- Users can list and delete their voice history and generated files from the learner app.
- Resumes and meeting data are personal data. Retention periods, consent text and a deletion flow must meet India's data-protection requirements before launch; confirm with counsel.

8. Integration catalogue

Twelve integration points connect the three areas and the outside world. Every cross-cloud call is idempotent (request ID), traced (one trace ID) and has a dead-letter path.

#	From	To	Mechanism	Purpose
I1	React apps	AWS API Gateway	HTTPS with Cognito JWT	All platform features
I2	React apps	GCP agent gateway	HTTPS with Cognito JWT	Resume writer, scheduler, chat with agents
I3	AWS platform services	GCP agent gateway	REST with JWT, signed	Platform asks an agent to do work
I4	GCP agent tools	AWS platform API	REST with a short-lived AWS role	Read profile and availability, write meetings and resumes
I5	AWS EventBridge	GCP Pub/Sub	Event forwarder, signed push	Profile, availability and order events reach agents
I6	GCP agents	AWS EventBridge	Signed webhook to API Gateway	Agent results such as resume ready and meeting booked
I7	Voice unit	GCP voice gateway	Streaming over mutual TLS	Audio up, reply audio down
I8	Pairing service	GCP device registry	Event, then registry write	Bind a device to its owner
I9	GCP fleet service	Voice unit	Pub/Sub command, signed image in Cloud Storage	Updates, reset, deactivation
I10	Scheduler tool	Google Calendar API	OAuth 2.0 per user	Check free-busy, create events and video links

#	From	To	Mechanism	Purpose
I11	Platform	Email or notification service	API	Invites, reminders, receipts
I12	Earn service	Payment provider	API and webhooks	Payouts and device purchases

Contract management: OpenAPI files and event JSON schemas live in `/packages/contracts`; both sides generate clients and validators from them, and CI fails on a breaking change.

9. Sequence: voice request

A spoken question reaches the agent through the GCP voice gateway, which fetches a user-scoped token from AWS and never exposes the device to the platform directly. Target: first spoken word within 2 seconds of the end of speech.

```
sequenceDiagram
  autonumber
  participant U as Student
  participant D as Pi unit
  participant G as Voice gateway (GCP)
  participant S as Speech services (GCP)
  participant A as Agent runtime (GCP)
  participant P as Platform API (AWS)
  U->>D: Hey unit, what is my next mentor meeting?
  D->>D: Wake word detected, start capture
  D->>G: Open stream over mutual TLS
  G->>G: Check device in registry, find owner
  G->>P: Ask for a user-scoped token for the owner
  P-->>G: Short-lived JWT
  D->>G: Audio frames
  G->>S: Streaming speech-to-text
  S-->>G: Transcript
  G->>A: Transcript, user JWT, session ID
  A->>P: GET next meeting with user JWT
  P-->>A: Meeting details
  A-->>G: Reply text
  G->>S: Text-to-speech
```

```
  S-->>G: Reply audio
  G-->>D: Stream reply audio
  D-->>U: Speaks the answer
  Note over A,P: Write actions need a spoken confirmation turn
```

- Steps 5 and 6 can run in parallel with audio capture so the token is ready when the transcript arrives.
- If the device is revoked or the owner link is missing, the gateway closes the stream and the unit says it needs to be set up again.
- Sensitive writes (payouts, account changes) are not done by voice alone; the agent asks the user to confirm in the app or with a PIN.
- The gateway logs device ID, owner ID, tools called and a trace ID, but not the audio.

10. Sequence: purchase and pairing

A unit is sold on the AWS platform, shipped with a printed claim code, and becomes usable only after the claim code binds it to the buyer's account and issues its certificate.

```
sequenceDiagram
  autonumber
  participant B as Buyer (learner app)
  participant C as Commerce service (AWS)
  participant PP as Payment provider
  participant E as Event bus (AWS to GCP)
  participant R as Device registry (GCP)
  participant F as Fleet and CA service (GCP)
  participant D as Pi unit
  B->>C: Place order for a voice unit
  C->>PP: Create payment
  PP-->>B: Hosted payment page
  B->>PP: Pay
  PP-->>C: Webhook payment succeeded
  C->>C: Mark order paid, create device record and claim code
  C-->>B: Order confirmed, shipping details
  C->>E: Publish order.paid
  E->>R: Create registry entry, status unclaimed
  Note over D: Unit arrives and is powered on
  D->>D: Serve setup page, user joins Wi-Fi
  B->>D: Enter claim code on the setup page
  D->>F: Pairing request with claim code and certificate request
  F->>R: Look up claim code, check unclaimed
```

```
  F->>C: Confirm claim for the logged-in owner
  C->>C: Bind device to owner, burn claim code
  C-->>F: Claim accepted
  F->>F: Certificate authority issues device certificate
  F->>R: Set status active, owner ID
  F-->>D: Certificate and configuration
  D-->>B: Pairing complete, confirmation chime
```

- The unit has a factory bootstrap credential that is valid only for the pairing endpoint; it is replaced by the per-device certificate at step 19.
- Claim codes are single use, expire, and are rate-limited per device to stop guessing.
- Refunds, returns and resale revoke the certificate and set the registry status to inactive; a new owner repeats pairing with a fresh claim code.
- Orders and payments stay entirely on AWS; GCP learns only the device ID and owner ID.

11. Sequence: resume writer

The resume tool drafts only from facts already on the student's profile, and nothing is saved until the student has reviewed and approved the draft.

```
sequenceDiagram
  autonumber
  participant S as Student (React app)
  participant G as Agent gateway (GCP)
  participant A as Agent runtime (GCP)
  participant R as Resume tool (GCP)
  participant M as Language model
  participant P as Platform API (AWS)
  participant F as Cloud Storage (GCP)
  S->>G: Request resume with target role and template
  G->>G: Validate JWT, apply rate limit
  G->>A: Start resume task
  A->>R: build_resume for user, role, template
  R->>P: GET profile, skills, courses, completed tasks
  P-->>R: Source data
  R->>M: Draft sections using only the source facts
  M-->>R: Structured draft as JSON
  R->>R: Check every claim against the source data
  R-->>A: Validated draft
  A-->>G: Draft ready
```

```
  G-->>S: Show draft in the editor
  S->>S: Review and edit
  S->>G: Approve final version
  G->>R: Render PDF from approved JSON
  R->>F: Store PDF
  R->>P: POST resume with JSON and file link
  P-->>S: Saved, download link, event resume.ready
```

- Model choice: Claude Sonnet 5.5 for the draft where quality matters, a cheaper Gemini model for formatting and checks, both behind the LLM adapter.
- Step 9 is a deterministic check: any employer, date or qualification not present in the source data is removed or flagged for the student.
- Drafts expire after a set period; only the approved version is kept on the platform.
- Jobs run asynchronously when the PDF is large, and the app polls the task status.

12. Sequence: mentor-mentee scheduling

The scheduler proposes ranked slots, the mentor confirms, and only then is the calendar event created. The platform holds the authoritative meeting record throughout.

```
sequenceDiagram
  autonumber
  participant E as Mentee (React app)
  participant A as Agent runtime (GCP)
  participant T as Scheduler tool (GCP)
  participant P as Platform API (AWS)
  participant C as Google Calendar
  participant N as Mentor (React app)
  E->>A: Book a session with this mentor
  A->>T: find_slots for mentee, mentor, duration
  T->>P: GET availability rules and existing meetings
  P-->>T: Availability for both
  T->>C: Free-busy for linked calendars
  C-->>T: Busy times
  T-->>A: Three ranked slots in each time zone
  A-->>E: Show slots
  E->>A: Choose a slot
  A->>T: hold_slot
  T->>P: POST meeting with status pending, idempotency key
  P-->>N: Notification of the request
  N->>P: Confirm or propose another time
```

```
  P->>T: Event meeting.confirmed
  T->>C: Create event and video link
  C-->>T: Event ID and link
  T->>P: PATCH meeting with link and event ID
  P-->>E: Confirmation and invite
  P-->>N: Confirmation and invite
  Note over T,P: Reminders at 24 hours and 1 hour; a reschedule repeats from
find_slots
```

- A held slot expires if the mentor does not respond within a set time, and the mentee is offered the next slot.
- If a calendar is not linked, the scheduler uses only platform availability and says so.
- Double booking is prevented by a unique constraint on mentor and time in Aurora, not by the agent.
- By voice the same flow runs, with the unit reading out the top slots and asking for a spoken choice.

13. Core data model

The platform owns all master data. GCP stores only agent state, generated files and the device registry.

Entity	Store	Owner	Key fields
User	Aurora	Platform	id, role, display name, time zone, consent flags
Profile and skills	Aurora	Platform	user id, headline, skills with level and evidence, education, links
Course, enrolment, progress	Aurora	Platform	course id, user id, status, completion date
Task and submission	Aurora	Platform	task id, reward, state, reviewer, submission link
Ledger entry	Aurora	Platform	account, amount, direction, reference, idempotency key
Mentor availability	Aurora	Platform	mentor id, weekly rules, exceptions, time zone

Entity	Store	Owner	Key fields
Meeting	Aurora	Platform	id, mentor, mentee, start, duration, status, link, calendar event id
Resume	Aurora and S3	Platform	id, user id, approved JSON, PDF link, template, created date
Order and device	Aurora	Platform	order id, user id, device id, claim code hash, status
Chat message and feed item	DynamoDB	Platform	conversation id, timestamp, sender, body
Agent session	Firestore	Agents	session id, user id, device id, turns, expiry
Draft resume and temporary files	Firestore and Cloud Storage	Agents	draft id, user id, expiry
Device registry entry	Firestore	Agents	device id, owner id, status, certificate fingerprint, last seen

Two rules keep the stores consistent: the platform is always the authority on conflicts, and every cross-service write carries an idempotency key so retries cannot duplicate meetings, payouts or orders.

14. Non-functional requirements, observability and CI/CD

Area	Target or approach
Availability	99.9% for platform APIs; agents degrade gracefully (the app still works without them)
Latency	Platform APIs under 300 ms at the 95th percentile; voice first spoken word within 2 seconds; resume draft under 30 seconds
Scale (design point)	50,000 users and 5,000 devices without redesign; see cost spreadsheet scenarios

Area	Target or approach
Resilience	Multi-AZ on AWS; regional Cloud Run on GCP; queues and dead-letter queues on every async path; retries with backoff
Region	AWS Mumbai (ap-south-1) and a nearby GCP region (Mumbai or Delhi) to keep voice latency low
Backup and recovery	Aurora point-in-time recovery, S3 versioning, Firestore export; recovery targets agreed per data class
Accessibility	WCAG 2.2 AA in every React app; the voice unit covers learners who prefer speech
Cost control	Budgets and alerts per cloud, per-feature model spend dashboard, model routing and caching

Observability

- OpenTelemetry in every Python service and the device agent; the trace ID is created at the edge and carried on every call and event.
- Logs to CloudWatch and Cloud Logging, with a shared dashboard for the three flows in sections 9 to 12.
- Alerts on error rate, latency, dead-letter queue depth, agent tool failures, and device offline rate.
- Model usage metrics per tool: tokens in and out, cost, latency, and user-visible failures.

CI/CD and environments

- Environments: development, staging, production, each in its own cloud accounts and projects.
- GitHub Actions builds containers, runs unit, contract and end-to-end tests, and deploys with Terraform; CI authenticates to AWS and GCP through OIDC.
- Contract tests run both directions across clouds on every change to `/packages/contracts`.
- Device images are built in CI, signed, tested on a hardware-in-the-loop Pi, and rolled out in rings (internal units, 5%, 25%, everyone) with automatic rollback.

15. Cost summary

Running cost is about $965 (₹93,393) a month for a pilot, $6,200 (₹6,00,036) for growth and $30,350 (₹29,37,273) at scale, at 96.78 rupees per dollar. These are planning figures, accurate to roughly plus or minus 30 percent; the full model with editable assumptions is the separate spreadsheet `Edtech_Cloud_Cost_Model.xlsx`.

Per month	Pilot	Growth	Scale
Active users	1,000	10,000	50,000
Voice units in use	100	1,000	5,000
AI and speech (USD)	$280	$2,800	$14,000
Infrastructure on AWS and GCP (USD)	$685	$3,400	$16,350
Total (USD)	**$965**	**$6,200**	**$30,350**
Total (INR)	**₹93,393**	**₹6,00,036**	**₹29,37,273**

Assumptions behind these figures:

- Models: Claude Sonnet 5.5 for the resume writer, Gemini 3.1 Flash-Lite for scheduling, Gemini 3 Flash for the voice reasoning step. Switching all workloads to Sonnet 5.5 raises Growth-stage AI and speech cost from about $2,800 to about $4,300 a month.
- Voice usage is 10 turns per unit per day, which costs about $2.60 (₹251) per unit per month. Heavy households at 30 turns a day cost about three times that, so price any voice subscription with usage in mind.
- Exchange rate is a mid-market quote from 10 October 2026; list prices for models are October 2026 figures. Infrastructure lines and the speech-to-text price are estimates to check against each provider's calculator.
- Not included: device hardware and shipping, payment-provider fees, calendar and email services, wake-word licence, staff, and tax.

16. Delivery phases, risks and open questions

Build in four phases so each one ships something usable. Durations depend on team size and are not estimated here.

Phase	Scope	Exit criteria
1. Foundations	Terraform for both clouds, CI/CD, Cognito, API Gateway, contracts package, learner and mentor app shells	A signed-in user can reach a stub endpoint on each cloud with one trace ID
2. Core platform	Learning, profiles, mentoring, availability, meeting records	Students can learn, mentors can publish availability, meetings are recorded
3. Agents	Agent gateway, resume writer, scheduler, event bridge between clouds	A student gets a reviewed resume PDF; a mentee books a mentor end to end
4. Earn and collaboration	Tasks, bounties, wallet, payouts, groups, chat	A task is completed, reviewed and paid with a reconciled ledger
5. Voice unit	Device image, voice pipeline, commerce and pairing, OTA	A purchased unit pairs and answers the voice flow in section 9

Key risks

Risk	Effect	Mitigation
Voice cost grows faster than revenue	Margin loss at scale	Route simple turns to a small model, cap session length, price per usage tier
Cross-cloud latency and egress	Slower agents, extra cost	Keep payloads small, pass file links, co-locate in nearby regions
Microphone privacy concerns	Low adoption, legal exposure	Local wake word, hardware mute, no stored audio by default, clear consent
Agent invents resume facts	Harm to students	Facts-only drafting, deterministic validation, mandatory student review
Device fleet support	Support load and returns	Remote diagnostics, signed OTA with rollback, device revocation

Risk	Effect	Mitigation
Payments and payouts compliance	Regulatory and fraud risk	Use a licensed provider, ledger reconciliation, KYC for payouts

Open questions

- Is the third product's hardware sourced and assembled by you, or by a partner?
- Which payment provider will handle device purchases and student payouts?
- Which languages must the voice unit support at launch (English only, or Hindi and others)?
- Should mentors and mentees be required to link Google Calendar, or is platform availability enough?
- What expected user and device volumes should the first release be sized for?